

 SINGLE-PAGE APPLICATION

TicketWave — Frontend Architecture

An architectural overview of the Angular 22 client: standalone components, signal-based reactivity, route guards, interceptors, and feature portals.

SECTION 1

Stack & Architecture Conventions

Modern standalone component design with fine-grained reactive primitives and strict naming standards.

| Technology Stack & Conventions



Core Framework Primitives

Angular 22: Built strictly with standalone components, completely omitting legacy `NgModules`.

Signal Reactivity: Uses `signal`, `computed`, and `effect` for application state, restricting RxJS exclusively to HTTP streams.



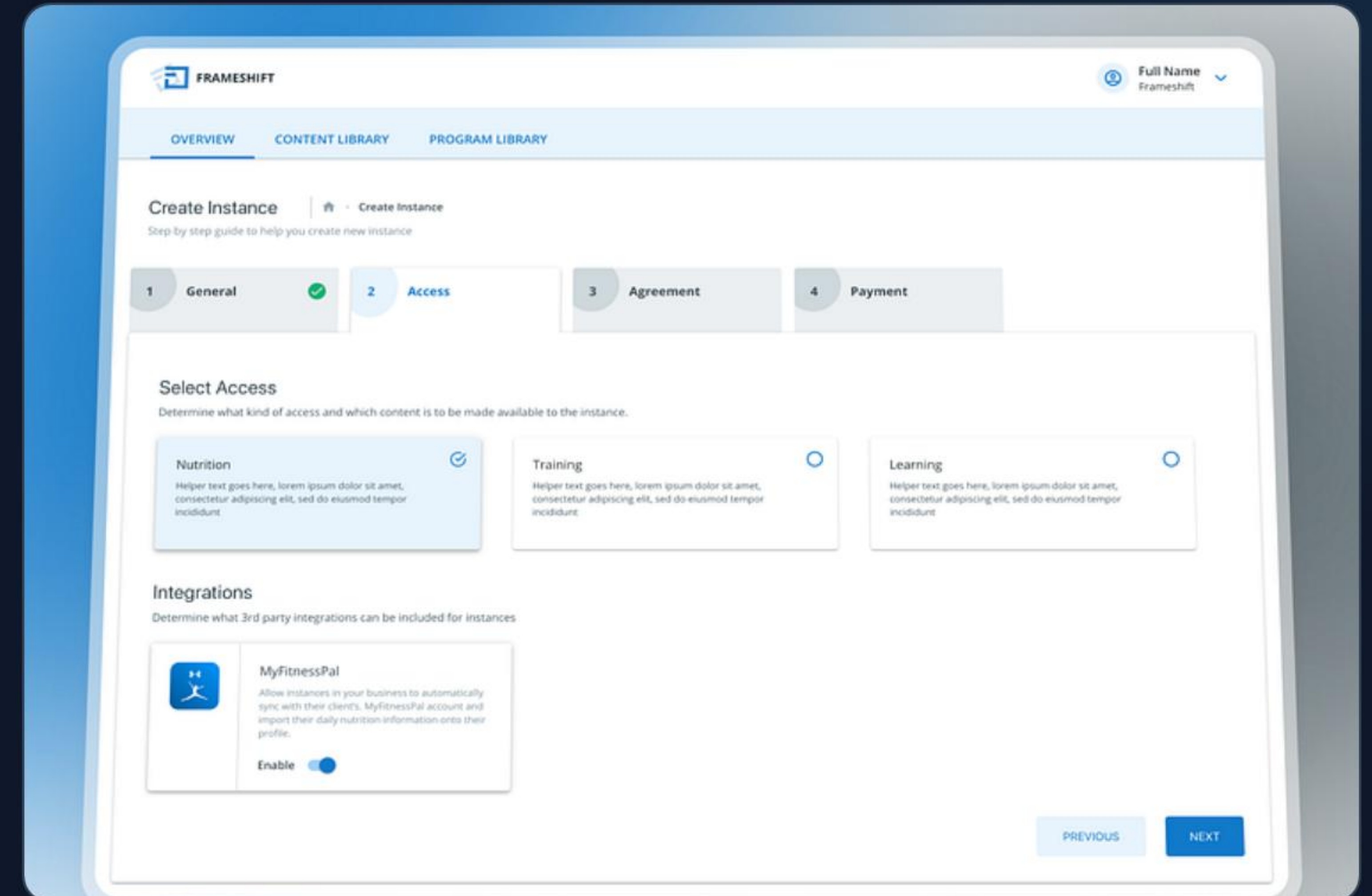
HTTP, Routing & Testing

Functional HTTP & Routes: Configured via `provideHttpClient` with functional interceptors and `provideRouter` with lazy `loadComponent`.

Automated Quality: Tested with Vitest for unit/component specs (57 files, 401 tests) and Playwright for E2E workflows.

Directory Architecture & Prefixes

-  **Core Directory** (`src/app/core`): Cross-cutting concerns containing route guards, interceptors, data models, services, and custom validators.
-  **Features Directory** (`src/app/features`): Modular, lazy-loaded user-facing domain areas (Search, Checkout, Portals).
-  **Shared Components** (`src/app/shared`): Reusable presentational elements prefixed with `tw-` (`tw-countdown` , `tw-qr-code`).



SECTION 2

Routing, Access Control & Auth

Multi-role route protection, deterministic landing resolution, and defensive token management.

Route Security Matrix

Path	Applied Guard	Target View & Role Scope
/	homeGuard	Redirects to role-specific landing screen
/search , /schedules/:id	Public Access	Schedule search & interactive seat selection
/checkout , /account	authGuard	Passenger details, promo code, profile & saved cards
/operator	roleGuard('OPERATOR')	Schedule manager, fleet editor & occupancy analytics
/support	roleGuard('SUPPORT')	Booking lookup, refund reviews & fee waivers
/admin	roleGuard('ADMIN')	User RBAC, partner credentials, webhooks & audit logs

Guard Hierarchy & Access Control



Auth Guard

Ensures user is authenticated. Saves target destination in `redirectTo` query params so post-login navigation resumes seamlessly.



Role Guard

Data-driven guard (`data: { role: 'OPERATOR' }`). Unauthenticated attempts redirect to login; unauthorized roles drop to `/search` .



Home Guard

Always returns a `UrlTree` . Evaluates roles deterministically (`ADMIN` → `SUPPORT` → `OPERATOR` → `/search`) preventing staff leakage.

| Authentication & State Management



AuthService Signal Store

Holds active JWT in a Signal backed by `localStorage` key `tw.accessToken`.

Exposes reactive computations: `isAuthenticated`, `username`, `roles`, `homePath`, and `isCustomer`.



Defensive Token Parsing

Token decoding is hardened against corrupt states. Malformed base64 or invalid JSON returns `null` rather than throwing runtime errors.

A corrupted token triggers clean logout and redirects safely without breaking the application shell.

SECTION 3

Interceptors, State & Portals

Global pipeline handlers, localized state isolation, and specialized user portals.

| HTTP Interceptors & State Scope



Functional Interceptors

authInterceptor: Attaches `Authorization: Bearer` automatically to outbound API calls.

errorInterceptor: Intercepts `401` errors to clear session & toast expiry. Normalizes `≥400` errors into auto-dismissing toasts.



Lean State Architecture

Zero reliance on heavy global stores like NgRx. State is strictly scoped:

In-Memory Drafts: `BookingDraftService` retains active seat holds during checkout flow and resets on page refresh to ensure valid hold states.

Customer Experience Portal

Live Seat Map & Holds

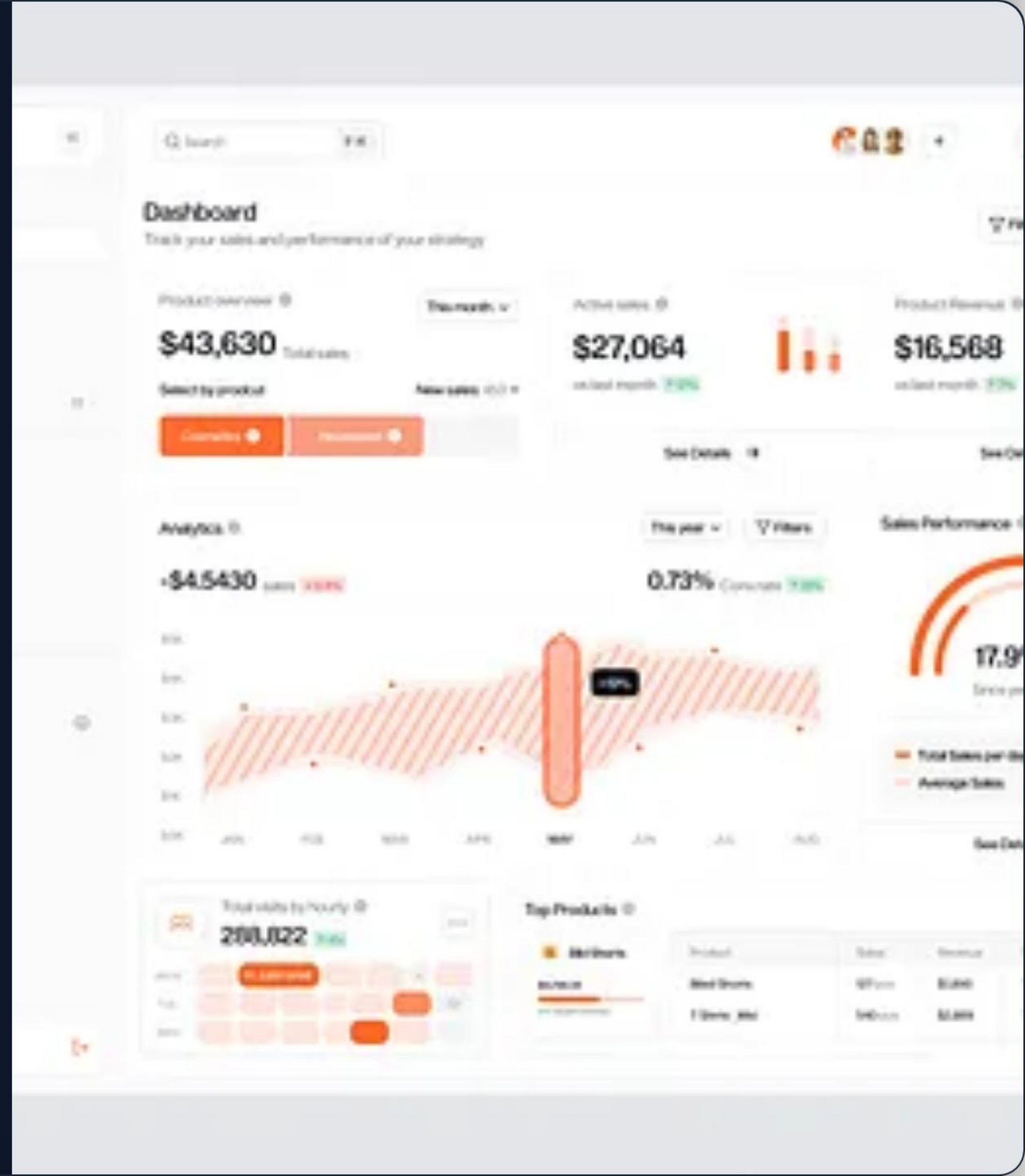
Interactive seat grid displaying real-time per-seat pricing. Integrated with **tw-countdown** to display remaining hold time.

Checkout & Self-Service

Client-side Luhn validation prevents invalid card submissions. Features full cancellation policy preview and PNR QR code generation.

Guest PNR Lookup

Allows unauthenticated users to find and manage bookings using PNR + email credentials.



| Operator, Support & Admin Portals



Operator Portal

Occupancy and revenue analytics dashboard, schedule manager, fleet editor (vehicles/drivers), and dynamic fare matrix management.



Support Panel

Customer booking lookup by PNR or email, refund request processing, and agent override fee waiver controls.



Admin Panel

User RBAC configuration, partner OAuth2 credentials and webhook management, promo code creation, and ledger reconciliation.

✓ TICKETWAVE FRONTEND VERIFIED

Questions & Discussion

57 spec files | 401 Vitest unit tests | Playwright E2E suites passing

```
cd frontend && npm start | proxy: http://localhost:8081
```


| Image Sources



<https://cdn.dribbble.com/userupload/10813298/file/original-87bde24e5795e2e3caade3b07bc03ca1.png?resize=752x&vertical=center>

Source: dribbble.com



<https://cdn.dribbble.com/userupload/16988431/file/original-10ba11b388d545a8399111d60719eb8f.jpg?format=webp&resize=400x300&vertical=center>

Source: dribbble.com